

0811_정지민



한 줄 요약

손 애니메이션에서 키프레임 사이를 자동으로 채우는 문제를 다룹니다.

"빠 길이가 늘어나는 문제"는 30줄짜리 고전 후처리로 완전히 해결되고, 딥러닝이 추가로 주는 것은 정확도 1~2%와 부드러움 14~25%입니다.

다만 제약을 아무것도 주지 않은 딥러닝 모델은 수식 세 줄짜리 보간보다 못합니다.

1. 무엇을 하는 프로젝트인가

주제 변경 안내

문제 정의

평가 지표 — 여러 축을 함께 봅니다

2. 데이터

겪은 문제 3가지

① 데이터에 작은 구멍이 잔뜩

② split 간 프레임레이트 불일치

③ 27개인 줄 알았던 촬영분이 실제로는 10개

최종 split (피험자 분리)

3. 실험 1 — 고전 기법 5종

결과 (test, 3.0초 갭 기준)

우승자 `smoothstep` 은 어떻게 동작하나

4. 실험 2 — 마스크 복원 Transformer

설계

1차 결과 — smoothstep에 완패

5. 실험 3 — 손실함수에 제약 추가

`w_bone` 스왑 — 트레이드오프가 거의 없음

`w_jerk` 스왑 — 너무 세면 모델이 붕괴

6. 정성 평가 — 손이 쪼그라들고 있었다

왜 쪼그라드나 — 지름길 때문

7. 실험 4 — 공정 비교 ⚠️ 결론이 바뀐 지점

문제 제기

뼈 길이 보정 (30줄)

결과 — 뼈 변형이 정확히 0.00%가 됩니다

같은 선상에서의 최종 비교

8. 결론

9. 한계

10. 다음 계획

11. 코드 및 문서

1. 무엇을 하는 프로젝트인가

주제 변경 안내

기존 제안 「가려진 손 추적」 (occlusion 구간 복원)에서 「손 모션 인비트위닝」으로 변경했습니다.

	기존 (가려진 손 추적)	변경 (인비트위닝)
입력	직전 N프레임만	앞 문맥 그리고 목표 키프레임
성격	외삽(예측)	내삽(보간)
정답 불확실성	큼	작음

앞뒤 양쪽이 주어지므로 문제가 훨씬 잘 정의되고, 기존 제안의 최대 리스크였던 "빈 구간에서 어떤 방법도 오차 급증"이 해소됩니다. 데이터 파이프라인은 거의 그대로 재사용했습니다.

문제 정의

[앞 문맥 10프레임] → [갭 = 생성 대상] → [목표 키프레임 1개]

애니메이터가 중요한 장면 몇 장만 그리면 그 사이를 자동으로 채우는 기술입니다. 기존 연구는 거의 전부 전신을 다루고 손은 무시하거나 고정 처리합니다. 손은 root 이동이 거의 없고 순수 국소 관절 운동이라 성격이 달라, 비어 있는 영역입니다.

평가 지표 — 여러 축을 함께 봅니다

지표	의미	좋은 값
MPJPE (mm)	생성된 관절이 실제와 얼마나 다른가	낮을수록
뼈 길이 변형 (%)	손뼈가 늘어나거나 줄었는가	3% 미만
jerk	동작이 덜덜 떨리는 정도	낮을수록



3%의 근거 — ground truth 자체가 촬영 오차로 약 3% 흔들립니다. 이보다 작은 차이는 데이터 노이즈에 묻혀 **판별 불가**입니다. 저울 눈금이 1g이면 0.3g을 못 재는 것과 같습니다.

왜 하나로 안 되는가: **hold** (마지막 자세 유지)는 실제 사람 자세를 복사하므로 뼈 변형이 정확히 **0.00%**지만 MPJPE는 최악입니다. 반대로 **cubic** 은 0.3초 갭에서 MPJPE 1위지만 3초에서 뼈를 38.6% 늘립니다. **하나만 보면 반드시 속습니다.**

2. 데이터

InterHand2.6M (Moon et al., ECCV 2020). 이미지는 쓰지 않고 관절 좌표 JSON만 사용 — **약 500MB** (전체 이미지는 365GB).

겪은 문제 3가지

① 데이터에 작은 구멍이 잔뜩

전체 관절값의 **4.33%**가 결측. 그대로 두고 자르니 2,400프레임 녹화가 **중앙값 5프레임 조각**으로 부서져 실험 자체가 불가능했습니다.

결측의 **89.6%**가 **2프레임 이하**라, 그 이하만 관절별 선형보간으로 메웠습니다. → segment 최대 길이 76 → **2,387프레임** 회복.

▼ 타당성 검증 (hold-out)

관측된 프레임을 일부러 지우고 같은 방식으로 복원해 실제와 비교했습니다.

지운 길이	복원 오차	같은 구간 실제 이동량
1프레임	1.23mm	7.76mm
2프레임	1.95mm	11.16mm

1프레임 복원 오차 1.23mm는 **GT 자체 노이즈(0.6~1.2mm)**와 같은 수준입니다. 또한 **평가 시에는 보간된 프레임을 제외**하므로 성적에 영향이 없습니다.

② split 간 프레임레이트 불일치

train은 초당 10장, val/test는 초당 30장이었습니다. 섞어 쓸 수 없어 **val/test를 3배 다운 샘플**해 10fps로 통일했습니다(위상 3개를 모두 사용해 프레임 손실 없음).

이에 따라 **갭 길이를 프레임이 아니라 시간으로 정의**합니다 → 0.3 / 1.0 / 2.0 / 3.0초

③ 27개인 줄 알았던 촬영분이 실제로는 10개

capture 10~26은 각각 300개 남짓 엔트리가 108개 시퀀스에 흩어져 있고 프레임 간격이 45(=1.5초)입니다. 동작이 아니라 **날장 정지 자세**라 이 연구에 쓸 수 없습니다. 코드 문제가 아니라 데이터가 실제로 없습니다.

최종 split (피험자 분리)

split	segment	프레임	피험자
train	1,570	96,373	0~8 (9명)
val	470	16,865	9 (1명)
test	703	31,851	10, 11 (2명)

평가 피험자는 학습에 전혀 등장하지 않습니다.

3. 실험 1 — 고전 기법 5종

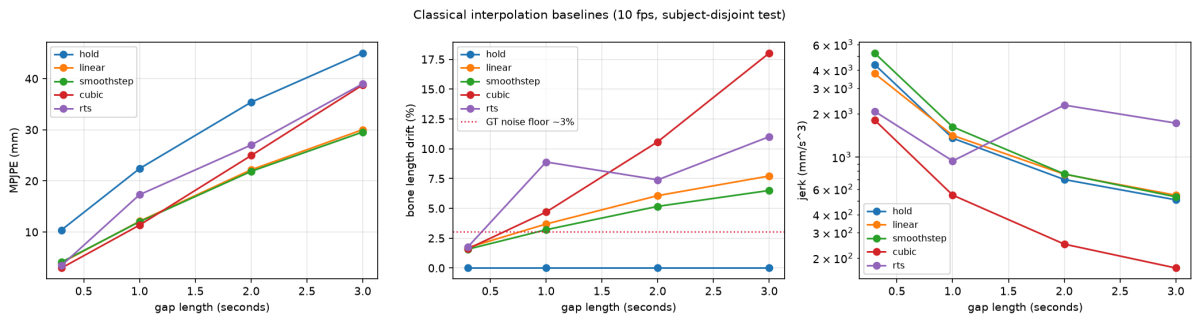
방법	하는 일	비유
hold	마지막 자세 유지	정지 화면
linear	시작-끝 직선 연결	자로 그은 선
smoothstep	부드럽게 가속·감속	천천히 출발해 천천히 도착
cubic	직전 속도를 이어받음	던진 공의 궤적
rts	물리 법칙(등가속도)	"이 속도로 가면 어디쯤?"

결과 (test, 3.0초 갭 기준)

방법	MPJPE (mm)	뼈 변형 (%)
hold	39.75	0.00
linear	30.71	4.51
smoothstep	30.91	3.75
cubic	69.85	38.64

방법	MPJPE (mm)	뼈 변형 (%)
rts	41.90	9.81

가장 단순한 **smoothstep** 이 우승했습니다.



갭 길이별 고전 기법 성능. 왼쪽부터 정확도 / 뼈 변형 / 떨림. cubic이 긴 갭에서 치솟는 것이 보입니다.

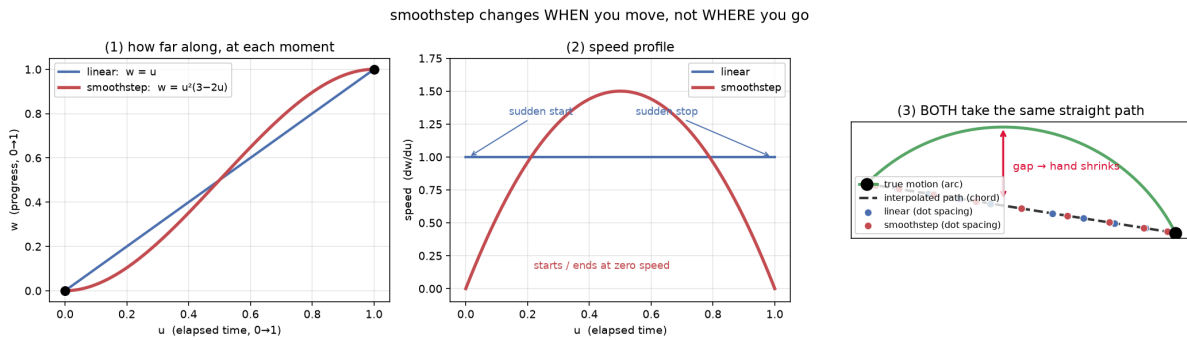
▼ 진 방법들에서 배운 것

cubic — 짧은 갭에선 1등인데 긴 갭에선 꼴찌. 직전 속도로 3초를 밀고 나가니 크게 오버슈트합니다. 심한 구간에서는 뼈가 **240% 늘어나** 손이 팔뚝만 해졌습니다. 구현 문제가 아님을 확인했습니다(문맥 길이 2/4/8 모두 같은 패턴, **not-a-knot** 경계조건은 217mm로 더 나쁨).

RTS — 아예 실패. 진단해보니 **목표 키프레임에 도달조차 못 합니다**(2초 갭에서 5.75mm 미달, smoothstep은 0.37mm). RTS는 추정 기법이라 "관측에 노이즈가 있고 모델이 옳다"를 전제하는데, 인비트위닝은 정반대로 **끝점이 정확하고 등가속도 모델이 틀렸습니다**. 손가락은 던진 공처럼 움직이지 않습니다.

우승자 **smoothstep** 은 어떻게 동작하나

진행률 u 를 그대로 쓰지 않고 $w = u^2(3-2u)$ 를 통과시킵니다. 조건 네 개를 만족하는 가장 낮은 차수의 다항식입니다 — 시작점에서 정확히 A, 끝점에서 정확히 B, 그리고 **양 끝에서 속도가 0**.



smoothstep은 '언제 얼마나 빨리'(속도)만 바꿉니다. '어느 길로 갈지'(경로)는 직선 그대로라 손 수축 문제를 못 고칩니다.



smoothstep이 이긴 이유는 잘해서가 아니라 "틀릴 여지가 없어서"입니다.
 cubic과 RTS는 각각 "이 속도로 계속 간다", "포물선을 그린다"는 **가정**을 합니다. 가정이 틀리면 크게 집니다. smoothstep은 가정을 하지 않아 특별히 잘하지도 않지만 크게 망하지도 않습니다.
 다만 **경로는 여전히 직선**입니다. 속도만 부드러워졌을 뿐 6절의 손 수축 문제는 그대로입니다.

4. 실험 2 — 마스크 복원 Transformer

설계

BERT 방식. 갭을 학습된 [MASK] 임베딩으로 치환하고 양방향 self-attention으로 복원합니다.

index 0..9	앞 문맥 10프레임	(관측)
index 10..9+L	갭	← [MASK]
index 10+L	목표 키프레임	(관측)



목표 키프레임의 위치가 갭 길이에 따라 달라지므로, 위치 인코딩만으로 모델이 "목표까지 몇 프레임 남았는지"를 압니다. 덕분에 갭 길이 4종에 대해 모델 하나만 학습하면 됩니다.
residual 설계: smoothstep 위의 *차이*만 예측합니다. 출력층을 0으로 초기화해 학습 시작 시점이 정확히 smoothstep이고, 끝점 연속성이 공짜로 보장됩니다.

파라미터 2.14M (`d_model=256`, `layers=4`).

1차 결과 — smoothstep에 완패

3.0초 갭	MPJPE	뼈 변형
smoothstep (수식 3줄)	30.91	3.75%
Transformer (214만 파라미터)	30.97	4.03%

갭 길이 전부에서 두 지표 모두 졌습니다.

원인은 단순했습니다. 학습할 때 "실제 위치와 가깝게 만들어라" 하나만 시켰고, "뼈 길이를 지켜라"는 말을 한 적이 없습니다. 모델은 시킨 것만 정확히 했습니다.



이것이 이 프로젝트에서 가장 견고한 발견입니다. 손의 해부학적 제약은 어딘가에 반드시 명시해야 합니다.

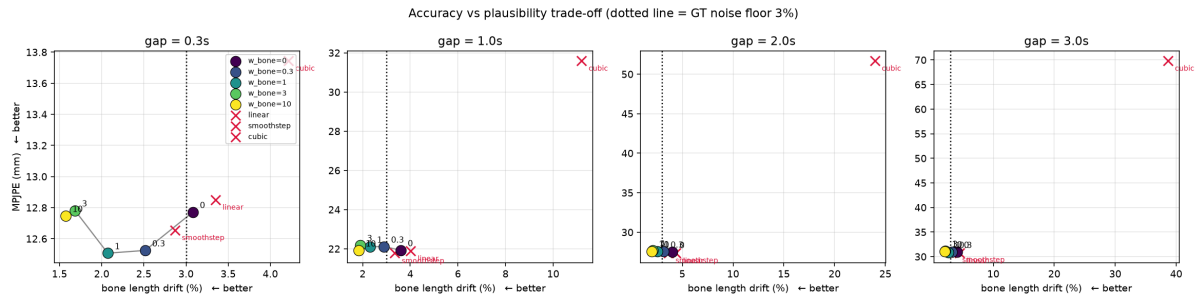
5. 실험 3 — 손실함수에 제약 추가

$\text{loss} = \text{위치오차} + w_{\text{bone}} \times \text{뼈길이오차} + w_{\text{jerk}} \times \text{떨림}$

`w_bone` 스윙 — 트레이드오프가 거의 없음

w_bone	MPJPE (3.0초)	뼈 변형 (3.0초)
0	30.97	4.03%
0.3	30.95	3.24%
1	30.88	2.60%
3	31.17	2.14%
10	31.08	2.06%

MPJPE는 1% 안팎에서 평평한데 뼈 변형만 절반으로 떨어집니다. 교환이 아니라 거의 공짜였습니다.



정확도-그럴듯함 트레이드오프. 가로축이 뼈 변형, 세로축이 MPJPE이며 왼쪽 아래로 갈수록 좋습니다. 점이 거의 수직으로 서 있는 것이 '거의 공짜'라는 뜻입니다.

w_jerk 스윙 — 너무 세면 모델이 붕괴

w_jerk≥5 에서 결과가 smoothstep과 정확히 겹쳤습니다. 모델이 실제로 출력하는 잔차를 재보니:

모델	잔차 크기	고쳐야 할 양 대비
w_jerk=1	3.02mm	11.0%
w_jerk=5	0.14mm	0.5%
w_jerk=20	0.24mm	0.9%

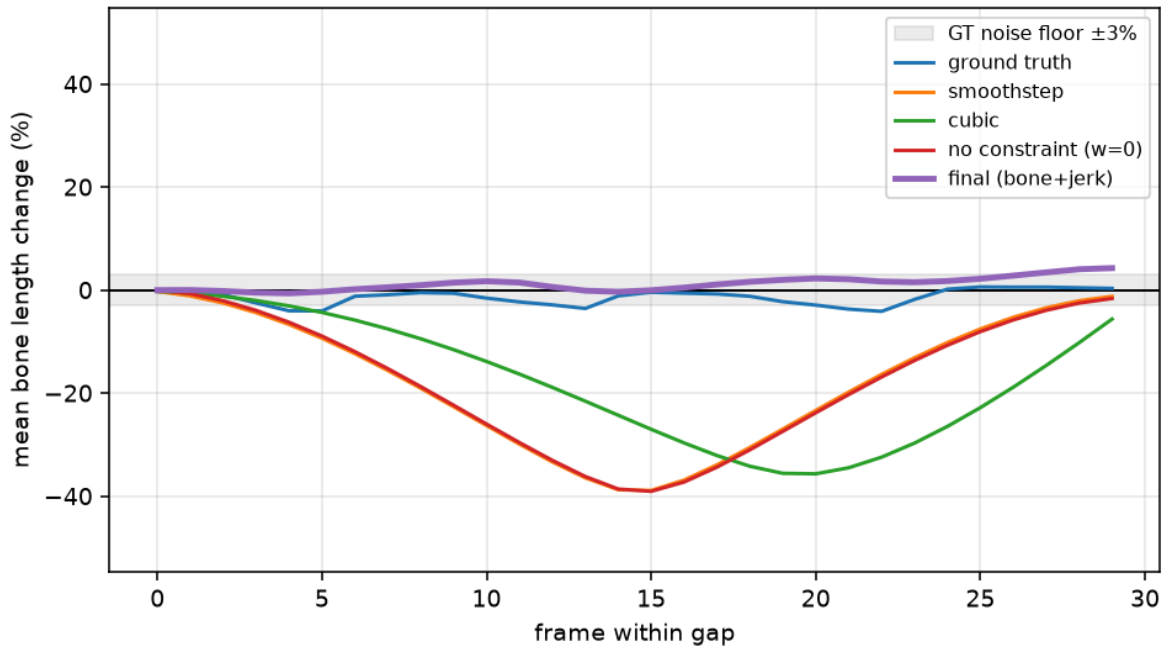
모델이 아무것도 출력하지 않는 법을 배웠습니다. residual 구조에서 출력층이 0 초기화이고 smoothstep은 이미 매우 부드러우므로, 떨림 벌점이 지배적이면 **가만히 있는 것이 국소 최적**이 됩니다. 사람으로 치면 "실수하지 마"라고 너무 압박해서 아무 시도도 못 하게 된 상황입니다.

6. 정성 평가 — 손이 쪼그라들고 있었다

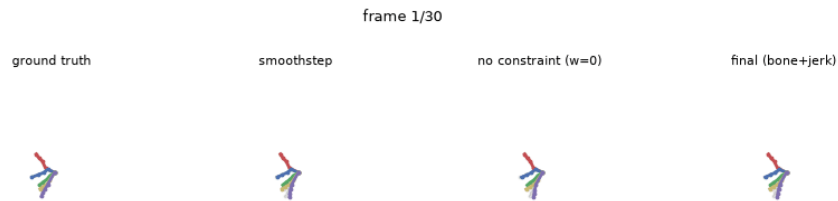
숫자만 보다가 실제로 그려봤더니, 뼈가 늘어난 게 아니라 줄어들고 있었습니다.

	갭 중앙에서의 뼈 길이
정답	±3% 유지
smoothstep	-39% (손이 61% 크기로 수축)
제약 없는 모델	-39%
최종 모델	±3% 유지

왜 쪼그라드나 — 지름길 때문



갭 구간 동안의 뼈 길이 변화. 회색 띠가 측정 하한($\pm 3\%$)입니다. 주황색(smoothstep)과 빨간색(제약 없는 모델)이 갭 중앙에서 -39%까지 내려갑니다. 보라색(최종 모델)은 정답과 거의 겹칩니다.



같은 구간 애니메이션. 회색이 정답이고 색상이 생성 결과입니다. smoothstep 손이 갭 중간에 한 번 오므라들었다 펴집니다.

손가락이 굽을 때 손끝은 **호(arc)** 를 그립니다. 그런데 시작-끝을 **직선으로 이으면** 그 호를 가로지르는 **지름길**로 갑니다. 활시위처럼요.

실제 움직임 : 손끝이 바깥쪽으로 **둥글게** (활대)
 직선 연결 : 안쪽으로 **질러감** (활시위) ← 손이 안으로 접힘

손목부터 손끝까지 **모든 관절이 동시에 지름길**을 타니 손 전체가 오므라듭니다. 양 끝은 실제 좌표에 고정되므로 **갭 중앙에서 최대**입니다.



이 관찰이 다음 절의 후처리 아이디어로 이어졌습니다 — 원인이 수축이라면 길이를 되돌리면 됩니다.

7. 실험 4 — 공정 비교 ⚠️ 결론이 바뀐 지점

문제 제기

앞선 비교는 학습 모델에만 제약을 주고 고전 기법에는 아무것도 안 붙였습니다. 같은 종류의 제약을 고전 기법에도 붙여야 공정합니다.

뼈 길이 보정 (30줄)

각 뼈의 방향은 유지하고 길이만 기준 자세에 맞춥니다. 손목에서 손끝 방향으로 순차 적용합니다. 기준 자세는 앞 문맥의 마지막 실측 프레임이며, 이는 학습 모델의 뼈 손실이 쓴 것과 동일한 정보입니다.

```
for child, parent in 손목에서_바깥_순서:
    방향 = 정규화(pred[child] - pred[parent])
    pred[child] = pred[parent] + 방향 * 기준_뼈길이
```

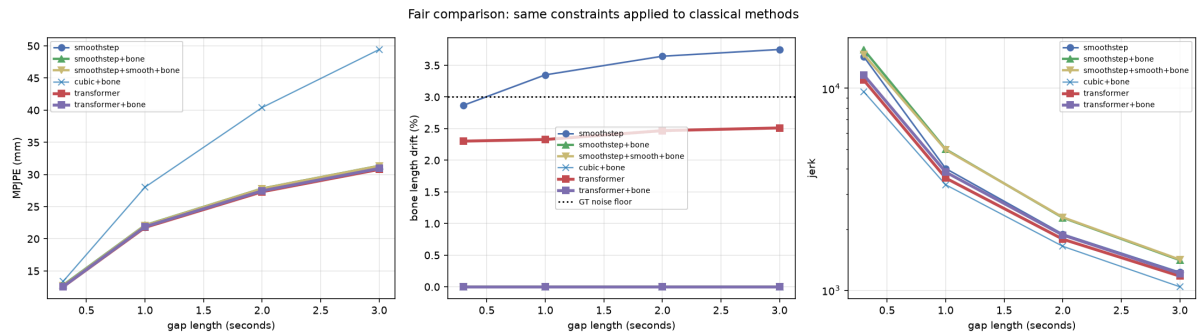
결과 — 뼈 변형이 정확히 0.00%가 됩니다

3.0초 갭	MPJPE	뼈 변형	jerk
smoothstep	30.91	3.75%	1231
smoothstep+보정	31.35	0.00%	1412
transformer	30.79	2.51%	1177
transformer+보정	30.94	0.00%	1213



뼈 변형 지표는 더 이상 변별력이 없습니다. 보정을 붙이면 어떤 방법이든 0%가 됩니다.

따라서 "학습 모델이 뼈 변형을 절반으로 줄였다"는 주장은 성립하지 않습니다. 30줄짜리 후처리가 더 완전하게(2.51% → 0.00%) 달성합니다.



공정 비교 — 고전 기법에도 뼈 길이 보정을 붙였을 때, 가운데 그래프에서 보정을 붙인 방법들은 전부 0%로 겹칩니다.

같은 선상에서의 최종 비교

갭	지표	smoothstep+보정	transformer+보정	차이
0.3초	MPJPE	12.79	12.55	-1.9%
	jerk	15561	11657	-25%
1.0초	MPJPE	22.12	21.88	-1.1%
	jerk	4994	3851	-23%
2.0초	MPJPE	27.82	27.46	-1.3%
	jerk	2290	1880	-18%
3.0초	MPJPE	31.35	30.94	-1.3%
	jerk	1412	1213	-14%

- 뼈 변형: **무승부** (둘 다 0.00%)
- MPJPE: 학습 모델 **1~2% 우위** — 작습니다
- jerk: 학습 모델 **14~25% 우위** — 이것이 실질적 기여입니다

▼ 부수 발견 3가지

- ① 보정은 MPJPE를 약간 악화시킵니다. smoothstep +1.4%, transformer +0.5%. 학습 모델이 보정으로 인한 손해가 더 적습니다 — 이미 뼈 길이에 가깝게 만들고 있어 보정이 덜 흔듭니다.
- ② cubic은 보정으로도 못 살립니다. 3초 갭에서 49.44mm. 보정은 형태를 고칠 뿐 위치가 엉뚱한 데로 날아가는 문제는 못 고칩니다.
- ③ Savitzky-Golay 스무딩은 효과가 없었습니다. 스무딩 후 뼈 보정을 걸면 보정이 스무딩을 되돌려놓는 것으로 보입니다. 순서를 바꾸거나 다른 방식이 필요합니다.

8. 결론

1. 고전 기법 중 최강은 **smoothstep**. 속도를 외삽하는 cubic과 물리 모델 RTS는 긴 갭에서 붕괴합니다.
2. 위치 오차만 학습한 Transformer는 **smoothstep에 완패**. 손의 제약은 어딘가에 반드시 명시해야 합니다.
3. 그 "어딘가"가 손실함수일 필요는 없습니다. 뼈 길이 문제는 30줄짜리 후처리로 **완전히 (0.00%)** 해결되며, 학습 모델의 손실함수 방식(2.51%)보다 낮습니다.
4. 같은 제약을 붙인 뒤에도 학습 모델이 이기지만 우위는 작습니다. 정확도 1~2%, 부드러움 14~25%.
5. 선형 보간의 실패는 손의 수축으로 나타납니다. 호가 아닌 현을 따라가기 때문이며, 갭 중앙에서 최대 -39%.
6. 제약이 과하면 모델이 붕괴합니다. **w_jerk $\geq$ 5** 에서 잔차 0.14mm로 베이스라인을 그대로 베깁니다.



실무적 함의

뼈 길이만 문제라면 **딥러닝은 불필요합니다**. 후처리로 충분하고 더 완전합니다. 동작의 **부드러움**이 중요하다면 학습 모델을 쓸 이유가 생깁니다.

9. 한계

한계	내용
피험자 9명	데이터 자체의 제약. 일반화 주장에 신중해야 합니다.
10fps	30fps 연속 데이터는 피험자 3명분뿐이라 프레임레이트를 낮춰 맞췄습니다.
학습 모델 우위가 작음	MPJPE 1~2%. 통계적 유의성 검정 미실시.
jerk 개선 원인 미분리	손실항 때문인지 모델 구조 때문인지 분리하지 않았습니다.
MANO 관절각 공간 미실험	관절각 공간은 뼈 길이가 구조적으로 보존됩니다.
자기관통 지표 미구현	메시가 필요해 뼈 길이 변형으로 대리했습니다.

10. 다음 계획

- ☐ **떨림(jerk)을 제대로 파기** — 이제 학습 모델의 유일한 실질적 기여. 고전 스무딩을 제대로 붙였을 때도 우위가 유지되는지 확인 (뼈 보정과 스무딩의 적용 순서 변경)
- ☐ **통계적 유의성 검정** — MPJPE 1~2% 차이가 우연이 아님을 보이기
- ☐ **MANO 관절각 공간 실험** — 제안서 연구질문 4
- ☐ **좌우 반전 증강** — 학습 데이터 2배, 과적합 완화

11. 코드 및 문서

▼ 실행 순서

```
python scripts/00_inspect_sequences.py --split train #
데이터 조사
python scripts/01_baseline_sweep.py #
고전 기법 비교
python scripts/02_tune_rts.py #
RTS 튜닝 (val)
python scripts/05_train_v2.py --w-bone 1 --w-jerk 1 --tag final
python scripts/04_final_comparison.py --models final
python scripts/06_tradeoff_curve.py --sweep bone #
트레이드오프 곡선
python scripts/08_qualitative.py --gap 30 #
정성 평가
python scripts/09_fair_comparison.py #
★ 공정 비교
```

▼ 코드 구조

```
src/
├─ data/
│   ├─ skeleton.py # 골격 토폴로지, 뼈 길이
│   └─ segment.py  # 시퀀스 분할, 결측 보간, 프레임레이트
              변환
│   └─ windows.py  # 윈도우 생성, split 준비
│       └─ dataset.py # 학습용 샘플러
└─ methods/
```

```

|   |— base.py           # ★ Infiller 인터페이스 (모든 방
법의 공통 규격)
|   |— interpolation.py   # hold / linear / smoothstep
/ cubic
|   |— rts.py            # RTS 스무더
|   |— transformer.py    # 마스크 복원 Transformer
|   |— postprocess.py    # 뼈 길이 보정, 스무딩
|— metrics/              # MPJPE, 뼈 변형, jerk, 학습 손
실
|— viz/skeleton.py       # 골격 시각화

```

모든 방법이 `Infiller.infill(past, target, L)` 하나를 구현하므로 평가 코드를 한 번만 쓰면 되고 방법 간 비교가 공정합니다.

▼ 실험 설계 규칙 (팀 공유)

- **갭 길이는 시간 단위로 정의합니다** (0.3 / 1.0 / 2.0 / 3.0초). 데이터가 10fps라 프레임 수로 쓰면 다른 fps와 비교가 안 됩니다.
- **하이퍼파라미터는 val에서 고르고 test는 마지막에 한 번만 짭니다.** 고전 기법(RTS의 `q`)에도 같은 규칙을 적용합니다.
- **평가는 실측 프레임에서만 합니다.** 결측을 메운 프레임을 정답으로 쓰면 보간 기법이 자기가 만든 답을 맞히는 순환 논증이 됩니다.
- **지표는 정확도와 그럴듯함을 함께 봅니다.** 어느 하나만 보면 반드시 속습니다.
- **베이스라인에도 같은 제약을 붙입니다.** 실험 4에서 배운 교훈입니다.

상세 문서는 저장소의 `docs/중간정리.md` (기술)와 `docs/쉽게 읽는 정리.md` (비전공자용)에 있습니다. 그림과 애니메이션은 `results/figures/` 에 있습니다.